

WINDOWS POWERSHELL ENCODED COMMAND INVESTIGATION

Project 4 — Controlled SOC Laboratory Investigation

Assessment: BENIGN / CONTROLLED LAB ACTIVITY — LOW SUSPICION

1. Executive Summary

This investigation examined a controlled Windows PowerShell process launched with **-EncodedCommand**. The objective was to determine what executed, identify the user and process context, correlate Sysmon and PowerShell telemetry, check for attributable network activity, and assess whether the execution showed evidence of persistence or other suspicious follow-on behavior.

The activity was assessed as **Benign / Controlled Lab Activity — Low Suspicion**. Encoded PowerShell and ExecutionPolicy Bypass are legitimate SOC detection signals and warranted investigation, but the surrounding telemetry was consistent with the intentionally controlled test.

2. Objectives

- Identify the encoded PowerShell process and process ID.
- Correlate the process to the user, parent process, executable path and integrity level.
- Use Sysmon Event ID 1 for process-creation evidence.
- Use PowerShell Operational Event ID 4104 for ScriptBlock visibility.
- Check TCP and UDP activity for the investigated PID.
- Confirm whether the process remained active.
- Distinguish execution evidence from evidence of malicious impact.

3. Environment

Endpoint: SOC-LAB-WIN11 (Windows 11 VM)

User: SOCAlyst\SOCAlyst2

PowerShell: Windows PowerShell 5.1

Sysmon: Installed and operational

Logs: Microsoft-Windows-Sysmon/Operational; Microsoft-Windows-PowerShell/Operational

4. Timeline

14:46:34 — Sysmon Event ID 1: PID 4720 created powershell.exe.

14:46:34 — PowerShell Event ID 4104: controlled ScriptBlock logged.

14:46:34 — Command line included -NoProfile -ExecutionPolicy Bypass -EncodedCommand.

15:09 — TCP check: no endpoints attributable to PID 4720.

15:10 — UDP check: no endpoints attributable to PID 4720.

15:11 — Process-state check: PID 4720 no longer present.

15:12–15:18 — Time-scoped 4104 checks confirmed the exact controlled ScriptBlock.

5. Process Creation Evidence

Sysmon Event ID 1 identified C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe as PID 4720 under SOCAlyst\SOCAlyst2. The event recorded High integrity and parent PID 7252, another PowerShell process.

The command line contained **-NoProfile -ExecutionPolicy Bypass -EncodedCommand**. These are meaningful SOC detection signals, but their presence alone is insufficient to classify the activity as malicious.

6. PowerShell ScriptBlock Evidence

PowerShell Operational Event ID 4104 captured the controlled ScriptBlock:

```
Write-Output 'SOC-LAB-PROJECT4-ENCODED-TEST'; Get-Date; Get-Process | Select-Object -First 3 Name, Id
```

The marker string was written using Write-Output. A subsequent file check found no marker file, which is expected because

Write-Output writes to the PowerShell output stream rather than creating a file.

Analytical caution: Event 4104 proves that the ScriptBlock content was logged; it does not independently prove that every output operation produced visible console output.

7. Network Investigation

TCP: A process-specific Get-NetTCPConnection query for PID 4720 returned no results. This is a point-in-time observation showing no active TCP endpoints attributable to the process during the check.

UDP: A process-specific Get-NetUDPEndpoint query for PID 4720 returned no results. This is likewise a point-in-time observation and does not prove that the process could never have communicated over UDP.

8. Process Lifetime

The final Get-Process query returned no process object for PID 4720. The encoded PowerShell process was therefore no longer running during the final state check, consistent with a short-lived execution.

9. Detection & Telemetry Assessment

- Sysmon Event ID 1 provided process creation, user, parent process, integrity and command-line context.
- PowerShell Event ID 4104 provided ScriptBlock visibility into the controlled encoded content.
- Native TCP and UDP queries provided process-scoped network checks.
- Negative network results should be described as point-in-time observations, not absolute historical proof.
- PID, timestamp, user, parent process and executable path should be correlated together.

10. Analyst Assessment

Verdict: BENIGN / CONTROLLED LAB ACTIVITY — LOW SUSPICION.

The execution contained characteristics that would reasonably trigger SOC investigation: PowerShell, ExecutionPolicy Bypass and EncodedCommand. However, the controlled payload was intentionally benign and correlated evidence showed a known Windows PowerShell executable path, controlled lab user, captured ScriptBlock content, no attributable TCP endpoints, no attributable UDP endpoints, no continuing process at final check, and no identified persistence, credential access, privilege escalation or suspicious child process.

11. MITRE ATT&CK; Mapping

T1059.001 — Command and Scripting Interpreter: PowerShell — PowerShell was the execution mechanism observed in the lab.

T1027 — Obfuscated/Compressed Files and Information — Encoded command content is relevant as a detection concept; this is not an assertion of adversary activity.

12. SOC Lessons

1. Encoded PowerShell is a lead, not a verdict.
2. Correlate process creation with ScriptBlock logging.
3. Use PID + timestamp + user + parent process together.
4. Check both process and network telemetry before assessing impact.
5. Document negative evidence carefully as point-in-time observations.
6. Separate what telemetry proves from what it merely suggests.

14. Conclusion

This investigation demonstrates a realistic SOC workflow for triaging an encoded PowerShell alert: identify the process, inspect the ScriptBlock through logging, correlate execution context, check for follow-on processes and network activity, and reach a proportionate conclusion based on the total evidence.

Final assessment: Benign / Controlled Lab Activity — Low Suspicion.

13. Evidence Screenshots

Initial lab evidence / controlled test preparation.

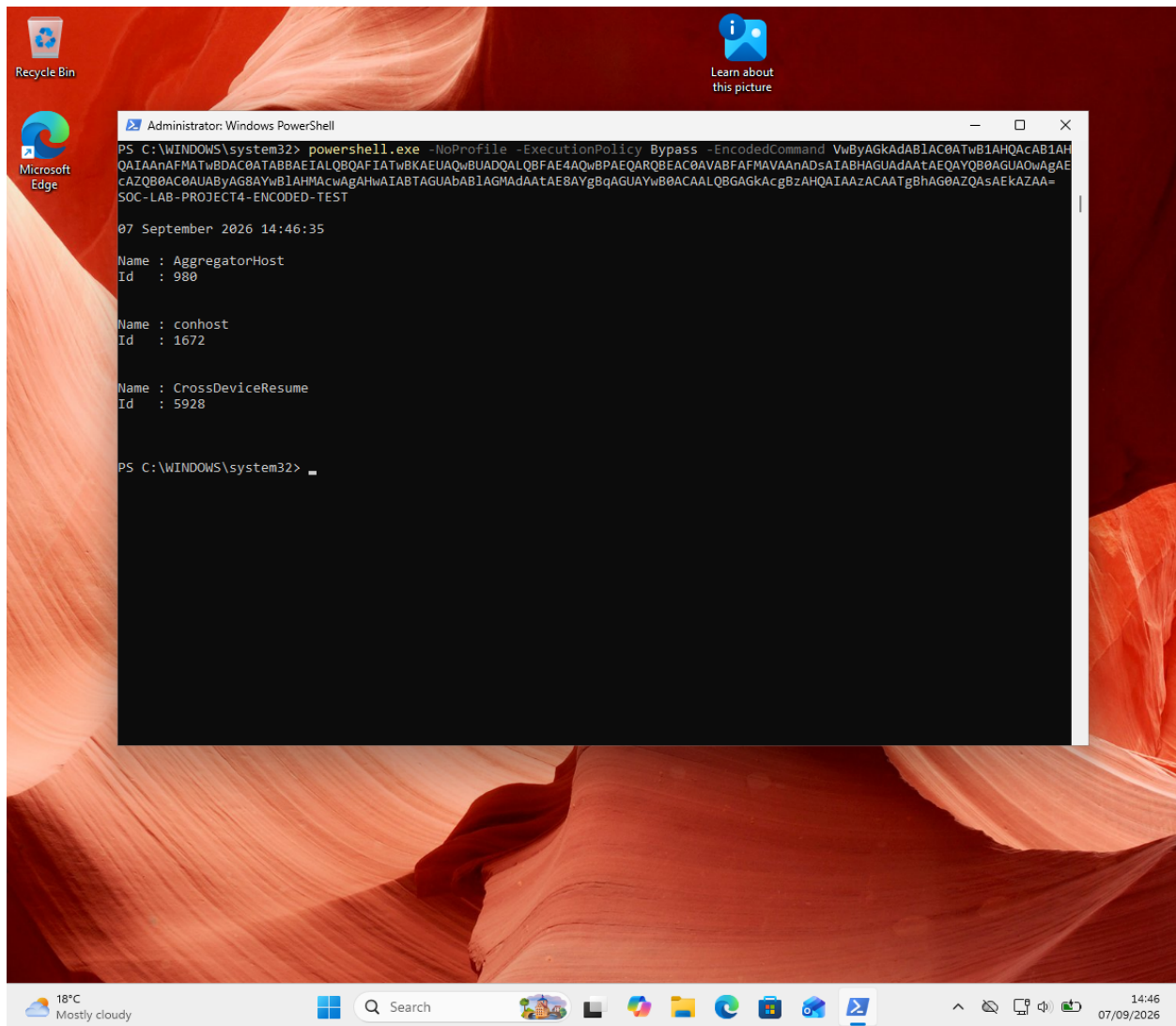
```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1; StartTime=(Get-Date).AddMinutes(-5)} |
>> Where-Object { $_.Message -match 'powershell.exe' } |
>> Select-Object TimeCreated, Id, Message -First 5

TimeCreated      Id Message
-----
-- Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1; StartTime=(Get-Date).AddMinutes(-10)} |>> Where-Object { $_.TimeCreated -ge (Get-Date '2026-09-07 14:17:25') -and $_.TimeCreated -le (Get-Date '2026-09-07 14:17:35') } |
>> Select-Object TimeCreated, Message |
>> Format-Listsystem32>

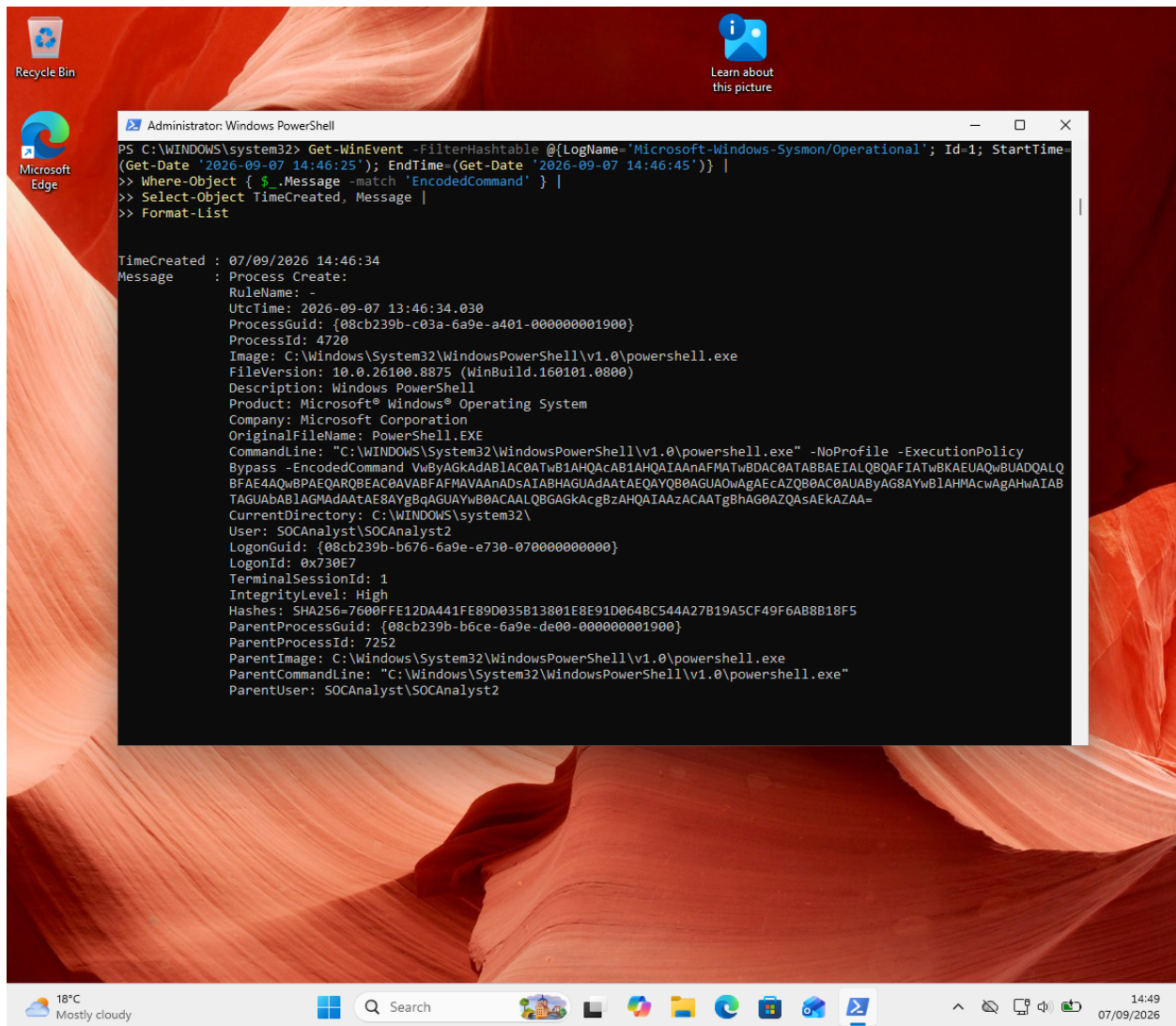
TimeCreated : 07/09/2026 14:17:30
Message      : Process Create:
               RuleName: -
               UtcTime: 2026-09-07 13:17:30.088
               ProcessGuid: {08cb239b-b96a-6a9e-6b01-000000001900}
               ProcessId: 5632
               Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
               FileVersion: 10.0.26100.8875 (WinBuild.160101.0800)
               Description: Windows PowerShell
               Product: Microsoft® Windows® Operating System
               Company: Microsoft Corporation
               OriginalFileName: PowerShell.EXE
               CommandLine: "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -ExecutionPolicy
               Bypass -Command "$u=$env:USERNAME; $h=$env:COMPUTERNAME; Get-Date; Get-Process | Select-Object -First 5
               Name,Id; Write-Output "SOC-LAB-PROJECT4-TEST"; Write-Output "User=$u Computer=$h""
               CurrentDirectory: C:\WINDOWS\system32\
               User: SOCAlyst\SOCAlyst2
               LogonGuid: {08cb239b-b676-6a9e-e730-070000000000}
               LogonId: 0x730E7
               TerminalSessionId: 1
               IntegrityLevel: High
               Hashes: SHA256=7600FFE12DA441FE89D035B13801E8E91D064BC544A27B19A5CF49F6AB8818F5
               ParentProcessGuid: {08cb239b-b6ce-6a9e-de00-000000001900}
               ParentProcessId: 7252
               ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
               ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
               ParentUser: SOCAlyst\SOCAlyst2

PS C:\WINDOWS\system32>
```

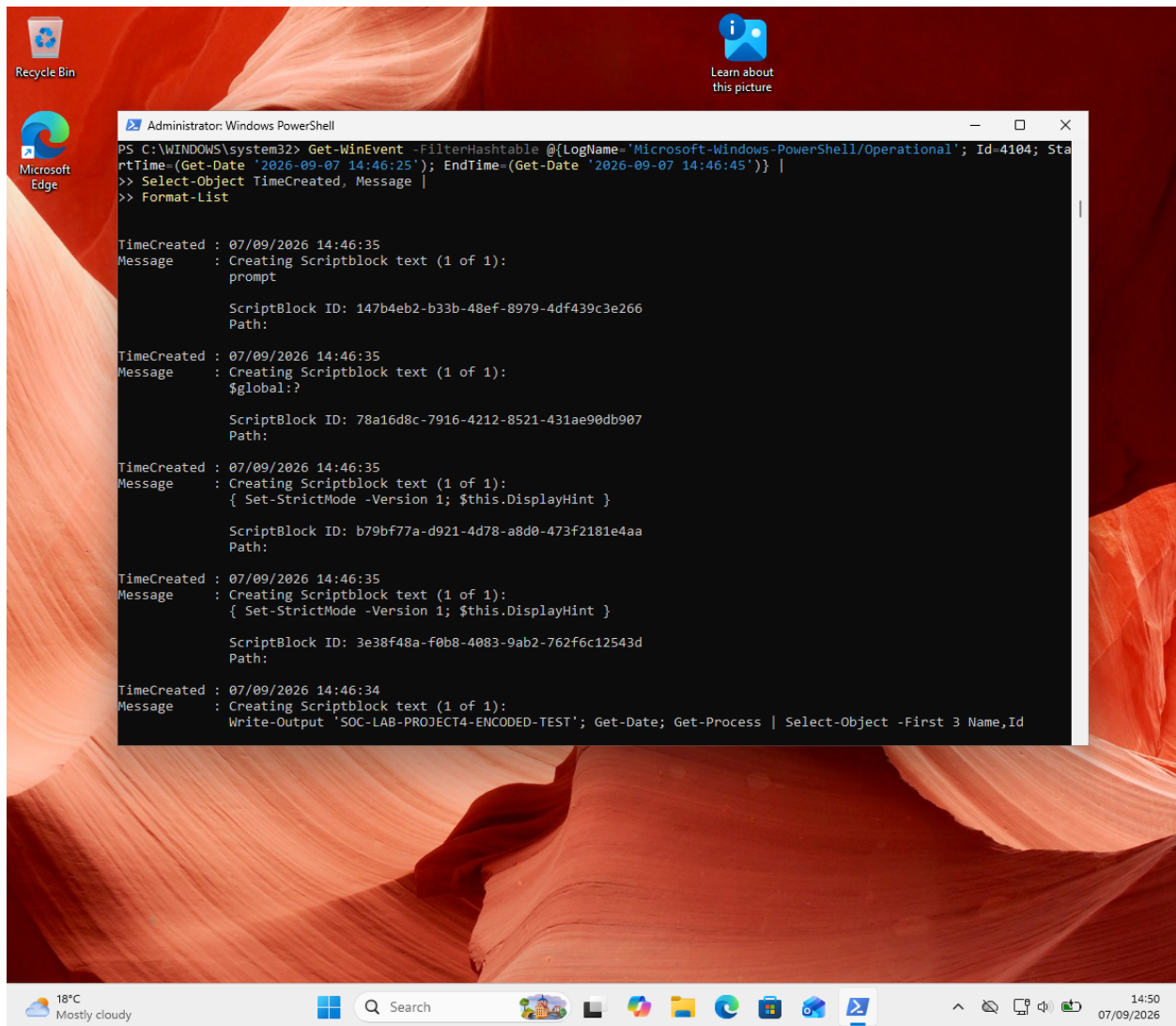
Encoded PowerShell execution and controlled marker output.



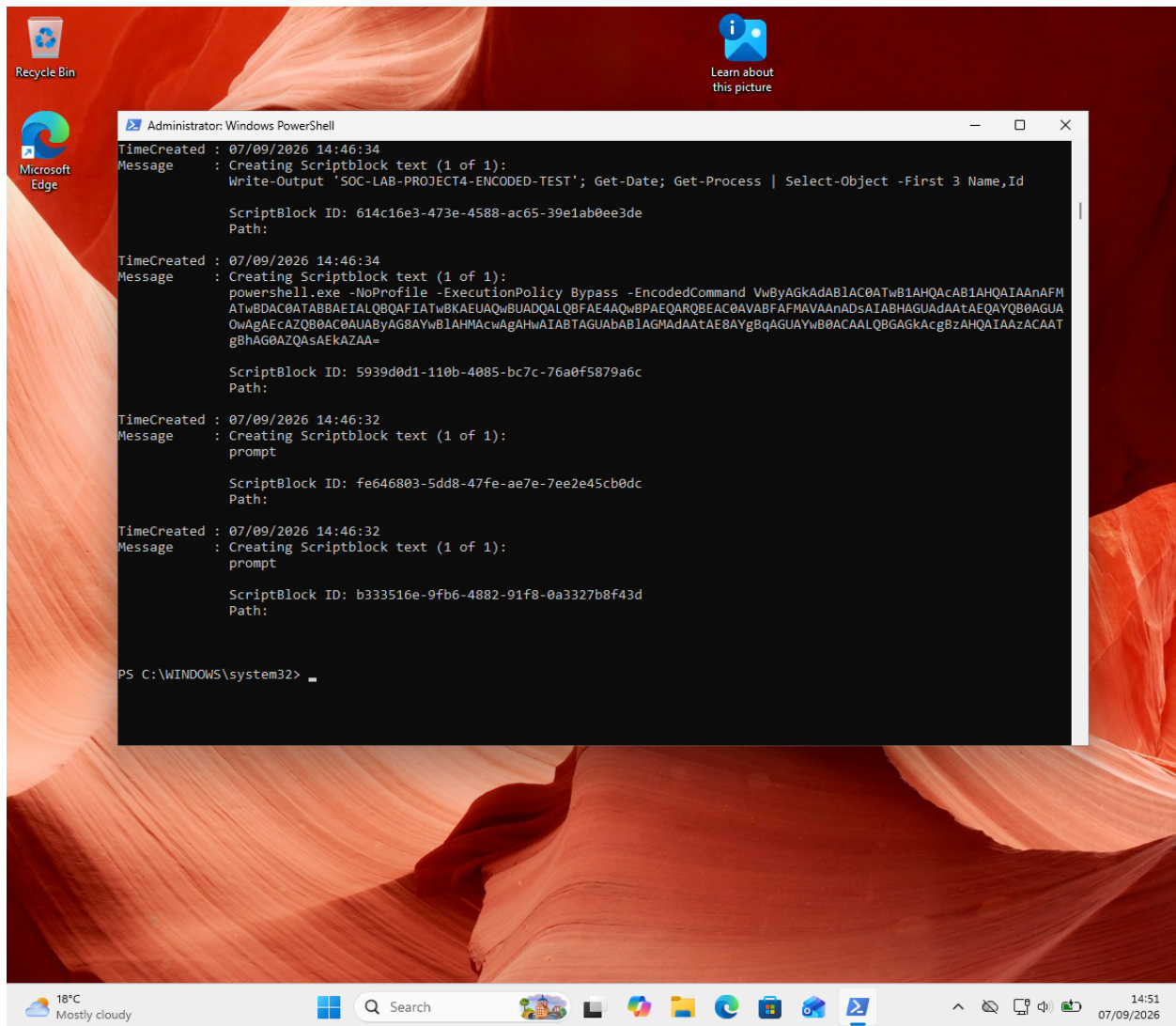
Sysmon process-creation evidence and process correlation.



PowerShell Operational logging / ScriptBlock evidence.



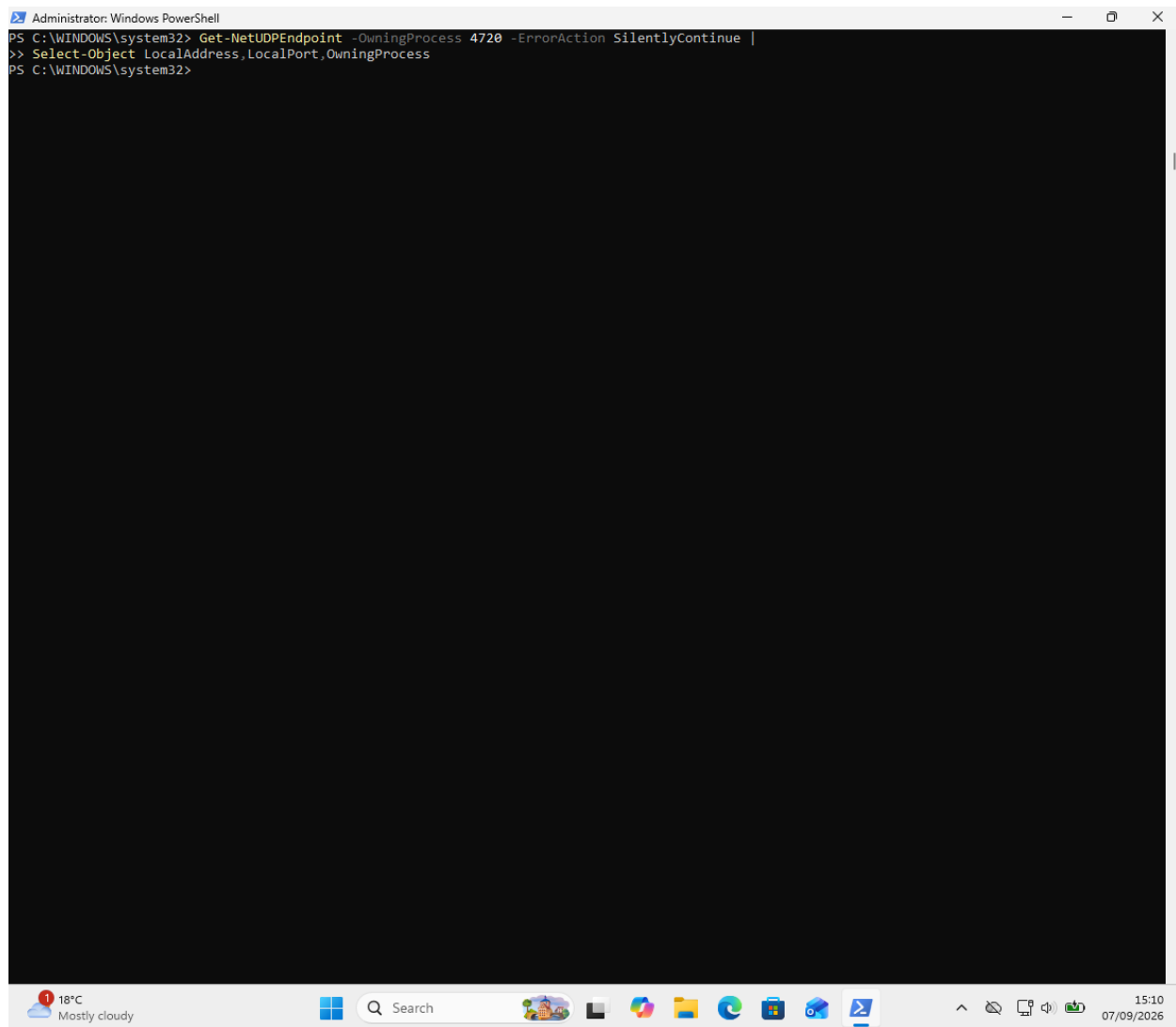
Additional process context from the controlled execution.



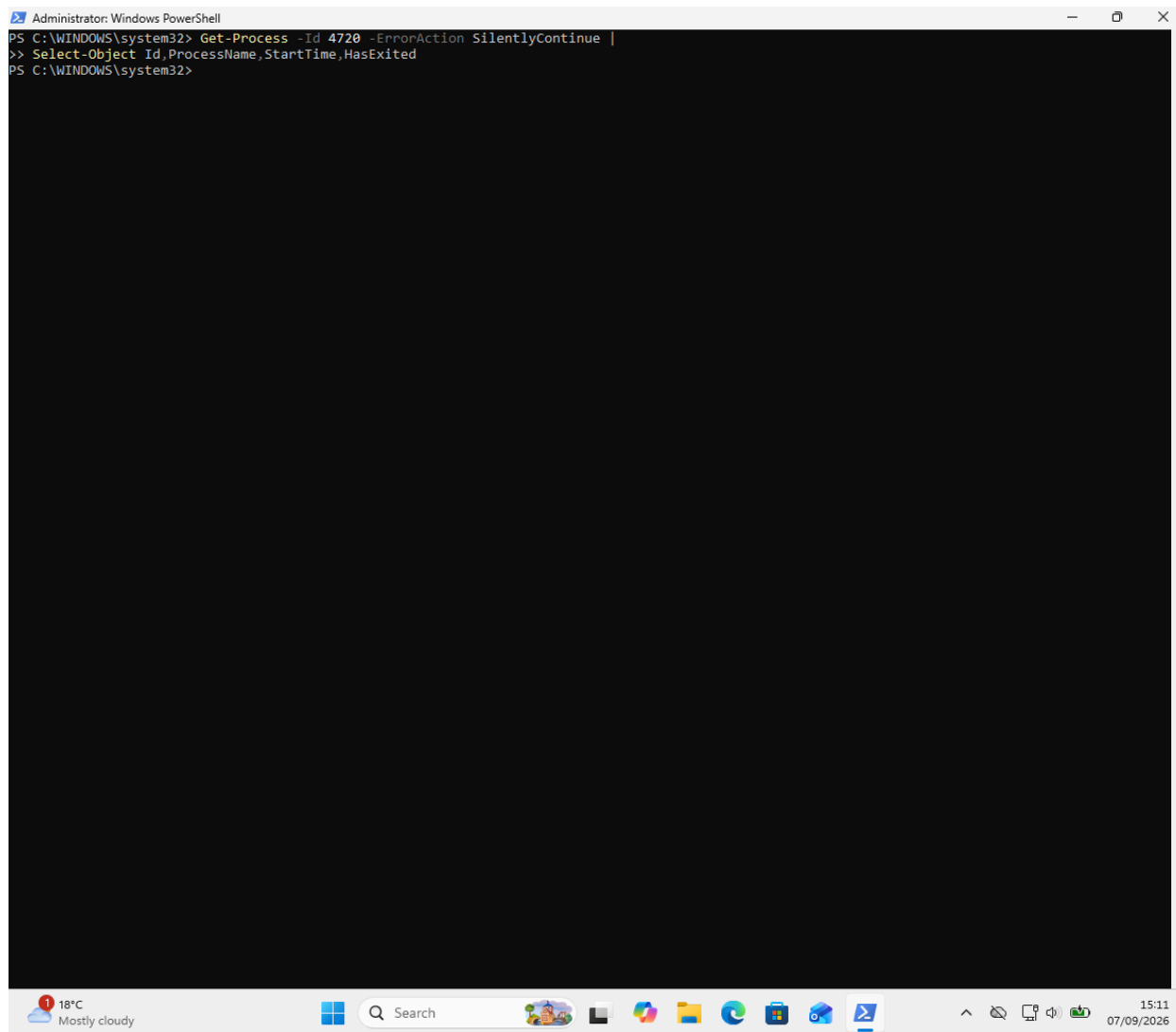
No active TCP endpoints attributable to PID 4720.

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-NetTCPConnection -OwningProcess 4720 -ErrorAction SilentlyContinue |
>> Select-Object LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwningProcess
PS C:\WINDOWS\system32>
```

No active UDP endpoints attributable to PID 4720.



PID 4720 no longer present at final process-state check.



PowerShell Event ID 4104 showing the controlled encoded ScriptBlock.

```
Administrator: Windows PowerShell
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104; StartTime=(Get-Date).AddHours(-1)} |
>> Where-Object { $_.Message -match 'SOC-LAB-PROJECT4-ENCODED-TEST' } |
>> Select-Object TimeCreated, Id, Message |
>> Format-List

TimeCreated : 07/09/2026 15:11:43
Id           : 4104
Message      : Creating Scriptblock text (1 of 1):
               Get-Item "C:\Windows\System32\SOC-LAB-PROJECT4-ENCODED-TEST" -ErrorAction SilentlyContinue |
               Select-Object FullName,Length,LastWriteTime

               ScriptBlock ID: bf916707-7e61-4a96-a413-bda9baef4a16
               Path:

TimeCreated : 07/09/2026 14:53:41
Id           : 4104
Message      : Creating Scriptblock text (1 of 1):
               { $_.Message -match 'SOC-LAB-PROJECT4-ENCODED-TEST' }

               ScriptBlock ID: 3bb35ccb-a255-49d0-a6b6-b410048f35e1
               Path:

TimeCreated : 07/09/2026 14:53:41
Id           : 4104
Message      : Creating Scriptblock text (1 of 1):
               $e = Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104; StartTime=(Get-Date '2026-09-07
               14:46:30'); EndTime=(Get-Date '2026-09-07 14:46:40')} | Where-Object { $_.Message -match 'SOC-LAB-PROJECT4-ENCODED-TEST' } |
               Select-Object -First 1
               [xml]$x = $e.ToXml()
               [PSCustomObject]@{
                 TimeCreated = $e.TimeCreated
                 EventID      = $e.Id
                 ProcessId    = $x.Event.System.Execution.ProcessID
                 ThreadId     = $x.Event.System.Execution.ThreadID
                 ScriptBlock  = ($e.Message -split "Creating Scriptblock text \\"(1 of 1)\:") [1].Trim()
               }

               ScriptBlock ID: 67e8a4fc-6ed7-4036-8473-9a83710af15e
               Path:

TimeCreated : 07/09/2026 14:46:34
Id           : 4104
Message      : Creating Scriptblock text (1 of 1):
               Write-Output 'SOC-LAB-PROJECT4-ENCODED-TEST'; Get-Date; Get-Process | Select-Object -First 3 Name,Id

               ScriptBlock ID: 614c16e3-473e-4588-ac65-39e1ab0ee3de
               Path:

PS C:\WINDOWS\system32>
```

Final time-scoped confirmation of the exact controlled ScriptBlock.

```
Administrator: Windows PowerShell

PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational'; Id=4104; StartTime=(Get-Date '2026-09-07 14:46:30'); EndTime=(Get-Date '2026-09-07 14:46:40')} |
>> Where-Object { $_.Message -match "Write-Output 'SOC-LAB-PROJECT4-ENCODED-TEST'" } |
>> Select-Object TimeCreated, Id, Message |
>> Format-List

TimeCreated : 07/09/2026 14:46:34
Id           : 4104
Message      : Creating Scriptblock text (1 of 1):
               Write-Output 'SOC-LAB-PROJECT4-ENCODED-TEST'; Get-Date; Get-Process | Select-Object -First 3 Name,Id

               ScriptBlock ID: 614c16e3-473e-4588-ac65-39e1ab0ee3de
               Path:

PS C:\WINDOWS\system32> 
```